

ABSTRACT

Solar energy is one of the most important renewable energy sources available today. Monitoring solar panel performance is essential for maximizing energy generation and ensuring efficient battery charging.

This project presents a **Solar Monitoring System Using Arduino** that continuously measures solar panel voltage, charging current, battery voltage, battery state of charge (SOC), power, and generated energy. The measured parameters are displayed on a 20×4 LCD screen in real time.

The system utilizes an Arduino Uno, ACS712 current sensor, voltage divider circuits, battery charging section, and LCD display. The project provides a simple and cost-effective solution for monitoring small solar installations.

INTRODUCTION

Solar photovoltaic systems convert sunlight into electrical energy. The performance of solar panels varies with solar irradiance, temperature, and load conditions.

Manual monitoring of solar systems is difficult and inefficient. Therefore, an automated monitoring system is required to:

- Measure solar voltage
- Measure charging current
- Monitor battery condition
- Calculate generated power
- Estimate generated energy

Arduino-based monitoring systems offer a low-cost and flexible solution.

LITERATURE REVIEW

Conventional solar monitoring systems generally employ:

- Digital multimeters
- Dedicated solar charge controllers
- IoT-based monitoring platforms
- SCADA systems

Most commercial systems are expensive for small-scale applications. Arduino-based monitoring systems provide:

- Low cost
- Easy implementation
- Real-time measurements
- Expandability

This project adopts an Arduino-based approach for efficient solar parameter monitoring.

OBJECTIVES

The main objectives are:

1. Monitor solar panel voltage.
2. Measure charging current.
3. Monitor battery voltage.
4. Estimate battery SOC.
5. Calculate output power.
6. Calculate generated energy.
7. Display parameters on LCD.
8. Provide charging indication.

HARDWARE REQUIREMENTS

Component	Quantity
Arduino Uno	1
ACS712 Current Sensor (5A)	1
20×4 LCD Display	1
Solar Panel (18V)	1
Battery (12V)	1
Diode 1N4007	1
Resistors	As required
LED Indicator	1
Connecting Wires	As required

SOFTWARE REQUIREMENTS

Software	Purpose
Arduino IDE	Program Development
Proteus 8 Professional	Simulation
Embedded C	Coding Language

CIRCUIT DESCRIPTION

The system consists of:

Solar Panel

Provides 18V DC supply.

ACS712 Current Sensor

Measures charging current flowing from the solar panel to the battery/load.

Voltage Divider Network

Resistors R1–R4 scale voltages to safe levels for Arduino analog inputs.

Arduino Uno

Acts as the central controller:

- Reads sensor values
- Performs calculations
- Updates LCD

Battery

Stores generated solar energy.

LCD Display

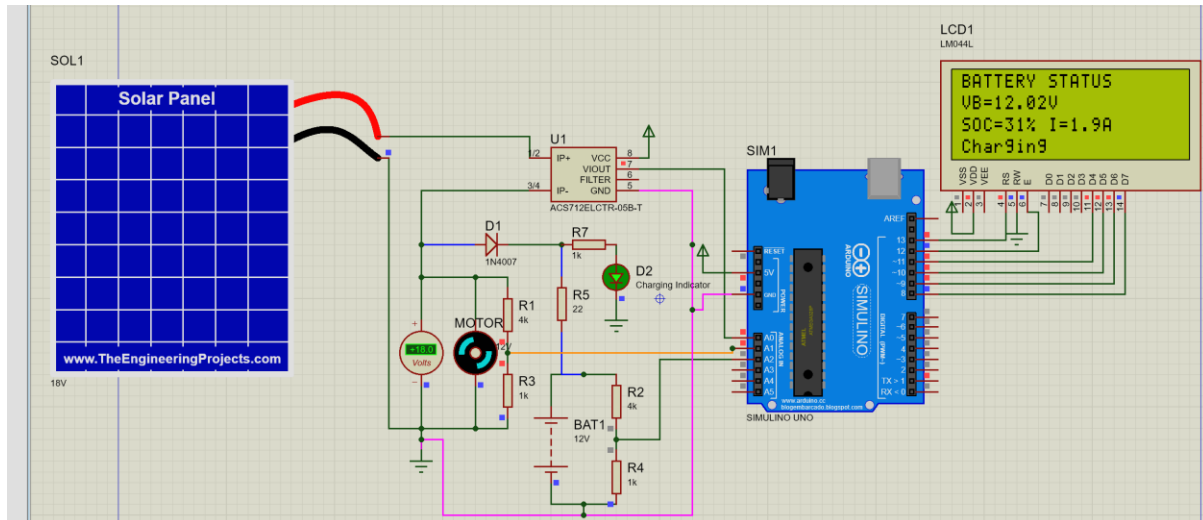
Displays:

- Solar voltage
- Current
- Battery voltage
- SOC
- Power

- Energy

LED Indicator

Indicates charging status.



WORKING PRINCIPLE

1. Solar panel generates DC power.
2. Current passes through ACS712 sensor.
3. Arduino reads:
 - Solar voltage
 - Battery voltage
 - Current
4. Voltage divider scales voltages to 0–5V.
5. Arduino calculates:
 - Power
 - Energy
 - Battery SOC
6. LCD displays monitored parameters.
7. LED glows during charging.

MATHEMATICAL CALCULATIONS

Solar Voltage

Voltage divider equation:

$$V_{in} = V_{out} \times \frac{R_1 + R_3}{R_3}$$

For:

$$R_1 = 4 \text{ k}\Omega$$

$$R_3 = 1 \text{ k}\Omega$$

Measured ADC voltage = 3.6V

Solar Voltage:

$$V_{\text{solar}} = 3.6 \times (4+1)$$

$$V_{\text{solar}} = 18\text{V}$$

Battery Voltage

For:

$$R_2 = 4 \text{ k}\Omega$$

$$R_4 = 1 \text{ k}\Omega$$

Battery ADC Voltage:

$$2.4\text{V}$$

Battery Voltage:

$$V_{\text{battery}} = 2.4 \times 5$$

$$V_{\text{battery}} = 12\text{V}$$

Current Calculation

ACS712 Sensitivity:

$$185 \text{ mV/A}$$

Current:

$$I = \frac{V_{\text{sensor}} - 2.5}{0.185}$$

Measured Current:

$$I = 1.9 \text{ A}$$

Power Calculation

$$P = VI$$

$$P = 18 \times 1.9$$

$$P = 34.2 \text{ W}$$

Energy Calculation

$$E = Pt$$

For:

$$P = 34.23 \text{ W}$$

$$t = 0.01175 \text{ h}$$

$$E = 0.402 \text{ Wh}$$

Battery SOC Estimation

SOC:

$$SOC = \frac{V_b - 11}{13 - 11} \times 100$$

For:

$$V_b = 12.02\text{V}$$

$$SOC \approx 31\%$$

ALGORITHM

1. Start.
2. Initialize LCD.
3. Read solar voltage.
4. Read battery voltage.
5. Read ACS712 current.
6. Calculate power.
7. Calculate energy.
8. Calculate SOC.
9. Display Solar Monitoring screen.
10. Display Battery Status screen.

11. Display Energy Generated screen.
12. Repeat continuously.

PROGRAM CODE

```
// include the library code:
#include <LiquidCrystal.h> //library for LCD

// initialize the library with the numbers of the interface pins
LiquidCrystal lcd(13, 12, 11, 10, 9, 8);

//Measuring Current Using ACS712
int sensitivity = 185; // use 100 for 20A Module and 66 for 30A Module
int adcvalue= 0;
int offsetvoltage = 2500; //Sensor Callibration
double Voltage = 0; //voltage measuring
double ecurrent = 0; // Current measuring
float power = 0;
float energy = 0;
float batteryVoltage = 0;
float batteryPercent = 30.0;
unsigned long previousMillis = 0;

void setup()
{
  Serial.begin(9600);
  lcd.begin(20, 4); // set up the LCD's number of columns and rows:
  lcd.setCursor(0,0);
  lcd.print(" THE BRIGHT LIGHT ");
  lcd.setCursor(0,1);
```

```

lcd.print(" SOLAR MONITORING      ");
previousMillis = millis();
}

void loop()
{
  Serial.println(analogRead(A0));
  // Measure The Votage *****
  // read the input on analog pin A1:
  int sensorValue = analogRead(A1);
  // Convert the analog reading (which goes from 0 - 1023) to a voltage (0 - 5V):
  float vol = (sensorValue * 5.0) / 1023.0;
  float voltage = vol * 5;

  // Battery Voltage Measurement (A2)
  int batteryADC = analogRead(A2);
  float batteryVol = (batteryADC * 5.0) / 1023.0;
  batteryVoltage = batteryVol * 5.0;

  // Measure The Current *****
  adcvalue = analogRead(A0); //reading the value from the analog pin A0
  Voltage = (adcvalue / 1024.0) * 5000; // Gets you mV
  ecurrent = abs((Voltage - offsetvoltage) / sensitivity);
  power = voltage * ecurrent;
  unsigned long currentMillis = millis();
  energy += power * ((currentMillis - previousMillis) / 3600000.0);
  previousMillis = currentMillis;
  String chargeStatus;

```



```

// Charging Status
if(voltage > batteryVoltage + 1.0)
{
    chargeStatus = "Charging";
    batteryPercent += (ecurrent * 0.1); // SOC increases while charging
    if(batteryPercent > 100)
    {
        batteryPercent = 100;
    }
}
else
{
    chargeStatus = "Standby";
}
batteryPercent = constrain(batteryPercent, 0, 100);

// ===== SCREEN 1 =====

lcd.clear();
lcd.setCursor(0,0);
lcd.print("SOLAR MONITORING");

lcd.setCursor(0,1);
lcd.print("V=");
lcd.print(voltage,1);
lcd.print("V");
lcd.setCursor(0,2);
lcd.print("I=");
lcd.print(ecurrent,1);
lcd.print("A");

```

```
delay(2000);
```

```
// ===== SCREEN 2 =====
```

```
lcd.clear();
```

```
lcd.setCursor(0,0);
```

```
lcd.print("ENERGY GENERATED");
```

```
lcd.setCursor(0,1);
```

```
lcd.print("E=");
```

```
lcd.print(energy,3);
```

```
lcd.print("Wh");
```

```
lcd.setCursor(0,2);
```

```
lcd.print("P=");
```

```
lcd.print(power,2);
```

```
lcd.print("W");
```

```
delay(2000);
```

```
// ===== SCREEN 3 =====
```

```
lcd.clear();
```

```
lcd.setCursor(0,0);
```

```
lcd.print("BATTERY STATUS");
```

```
lcd.setCursor(0,1);
```

```
lcd.print("VB=");
```

```
lcd.print(batteryVoltage,2);
```

```
lcd.print("V");
```

```
lcd.setCursor(0,2);
```

```
lcd.print("SOC=");
```

```
lcd.print(round(batteryPercent));
```

```

lcd.print("% ");
lcd.print("I=");
lcd.print(ecurrent,1);
lcd.print("A");
lcd.setCursor(0,3);
lcd.print(chargeStatus);
delay(2000);
}

```

SIMULATION RESULTS

Screen 1: Solar Monitoring

Observed:

- Solar Voltage = 18.0 V
- Current = 1.9 A

Screen 2: Battery Status

Observed:

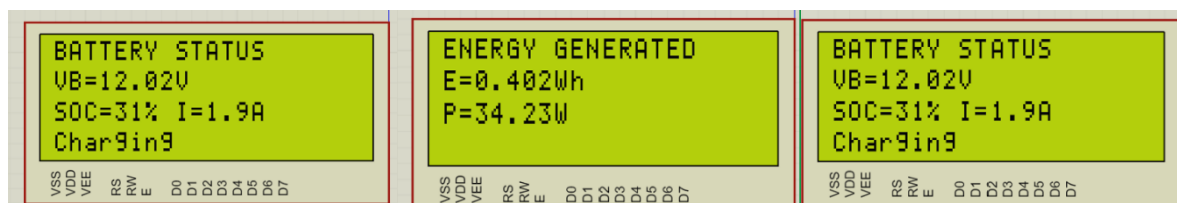
- Battery Voltage = 12.02 V
- SOC = 31%
- Charging Status = Charging

Screen 3: Energy Generated

Observed:

- Power = 34.23 W
- Energy = 0.402 Wh

The simulation successfully monitored all required solar parameters.



ADVANTAGES

- Low-cost implementation
- Real-time monitoring
- Easy to expand
- Simple hardware
- Accurate measurements
- User-friendly display

LIMITATIONS

- Simulation-based implementation
- SOC estimation is approximate
- No wireless communication
- Limited data storage

APPLICATIONS

- Solar home systems
- Solar street lighting
- Solar battery charging stations
- Educational laboratories
- Renewable energy projects

FUTURE SCOPE

- IoT integration using ESP8266/ESP32
- Mobile application monitoring
- Cloud data logging
- MPPT integration
- GSM-based alert system
- Wireless monitoring dashboard

CONCLUSION

The **Solar Monitoring System Using Arduino** was successfully designed and simulated using Proteus. The system accurately measures solar voltage, charging current, battery voltage, battery state of charge, power, and generated energy. The monitored parameters are displayed on a 20×4 LCD in real time.

The project demonstrates an effective, economical, and reliable solution for monitoring small solar photovoltaic systems and can be further enhanced using IoT technologies for remote monitoring and control.

REFERENCES

1. [Arduino Official Documentation](#)
2. [ACS712 Current Sensor Datasheet \(Allegro\)](#)
3. [Proteus Design Suite](#)
4. [LiquidCrystal Library Documentation](#)